

RECURSION



Recursion

1. A function, which calls itself either directly or indirectly through another function.
2. A recursive function is called to solve a problem.
3. The function actually knows how to solve only the simplest case(s), or so-called **base case(s)**
4. If the function is called with the **base case** it *simply returns the result or in some cases do nothing.*

Recursion

5. If the function is called with a more complex problem (A), the function divides the problem into **two** conceptual pieces (B and C).

These two pieces are a piece that the function **knows how to do** (B – a base case) and a piece that the function **does not know how to do** (C).

The latter piece (C) must resemble the original problem, but be a slightly simpler or slightly smaller version of the original problem (A).

Recursion Guidelines

- The definition of a recursive method typically includes an `if-else` statement.
 - ▣ One branch represents a base case which can be solved directly (without recursion).
 - ▣ Another branch includes a recursive call to the method, but with a “simpler” or “smaller” set of arguments.
- Ultimately, a base case must be reached.

Rules of Recursion

- ❑ Bad use of recursion, causes a huge amount of redundant work being performed, violating a major rule of recursion.

Rules of Recursion:

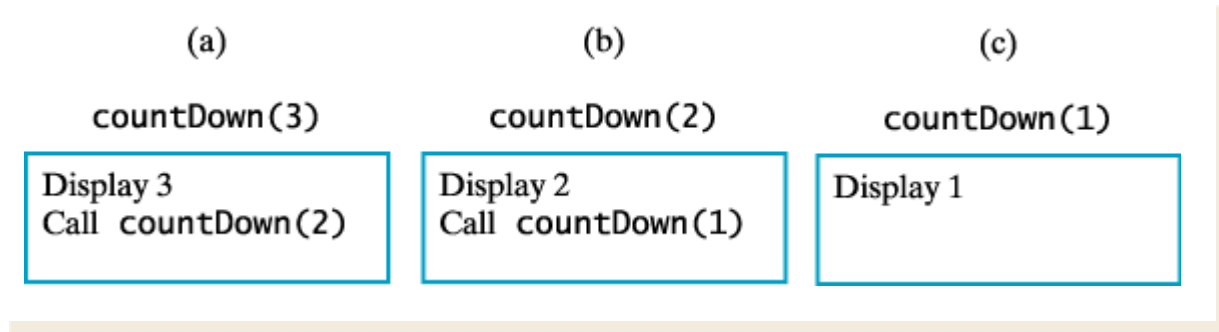
1. **Base Case:** You must always have some base cases, which can be solved without recursion.
2. **Making Progress:** Recursive call must always be to a case that make progress toward a base case.

Infinite Recursion

- ❑ If the recursive invocation inside the method does not use a “simpler” or “smaller” parameter, a base case may never be reached.
- ❑ Such a method continues to call itself forever (or at least until the resources of the computer are exhausted as a consequence of *stack overflow*).
- ❑ This is called *infinite recursion*.

Tracing a Recursive Method

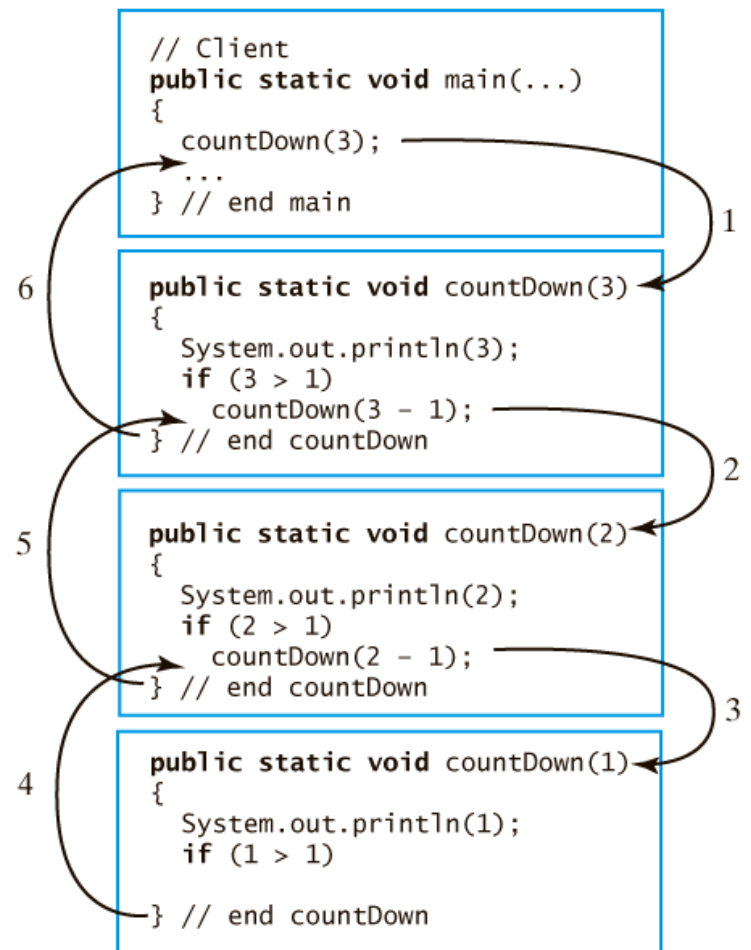
□ **Given:** **public static void** `countDown(int integer)`
 { `System.out.println(integer);`
 if (`integer > 1`)
 `countDown(integer - 1);`
 } // end `countDown`



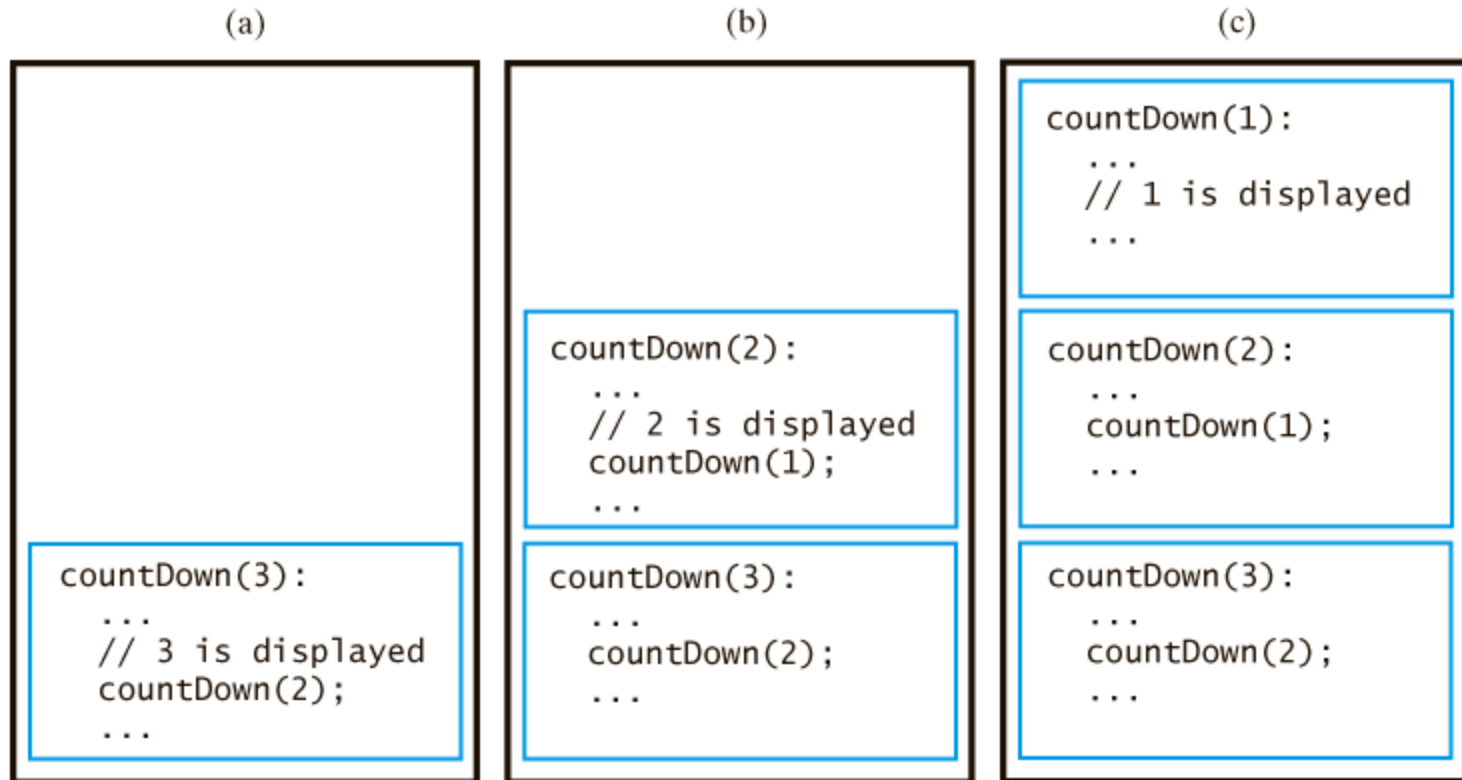
The effect of method call `countDown(3)`

Tracing a Recursive Method

Tracing the recursive
call **countDown (3)**

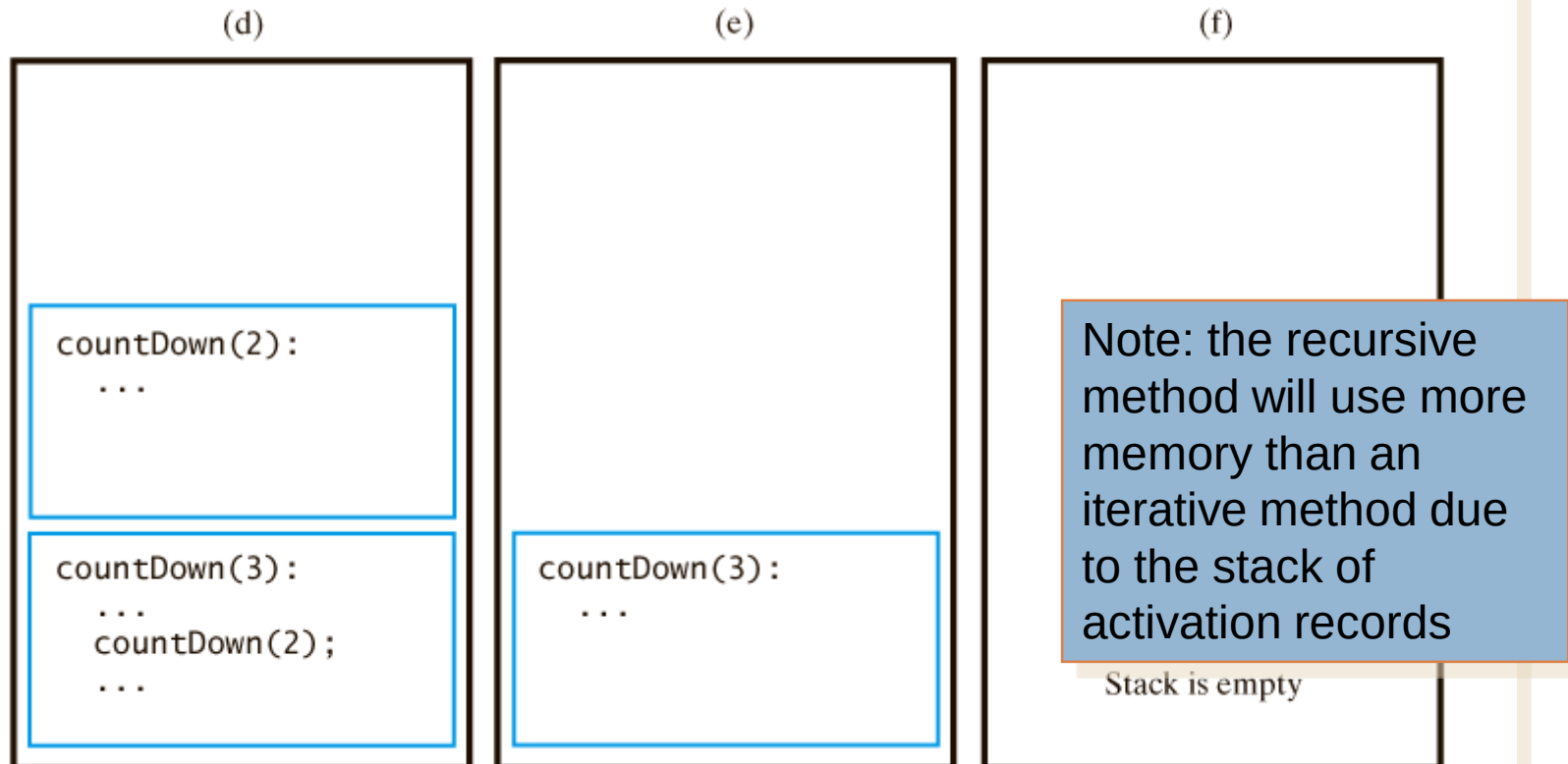


Tracing a Recursive Method



The stack of activation records during the execution of a call to `countDown(3)`... continued →

Tracing a Recursive Method



The stack of activation records during the execution of a call to `countDown(3)`

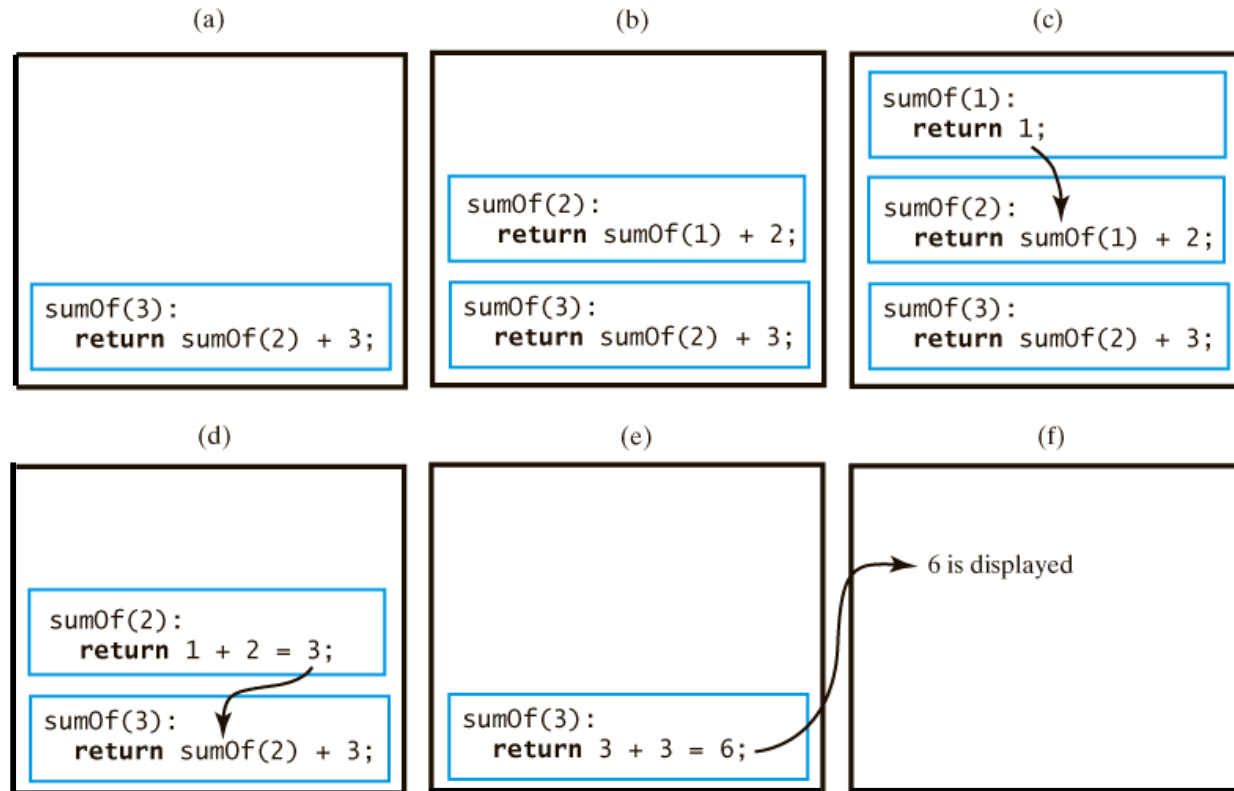
Recursive Methods That Return a Value

- Task: Compute the sum

$1 + 2 + 3 + \dots + n$ for an integer $n > 0$

```
public static int sumOf(int n)
{
    int sum;
    if (n == 1)
        sum = 1; // base case
    else
        sum = sumOf(n - 1) + n; // recursive call
    return sum;
} // end sumOf
```

Recursive Methods That Return a Value



The stack of activation records during the execution of a call to `sumOf(3)`

Recursion vs. Iteration

- Any recursive method can be rewritten without using recursion.
- Typically, a loop is used in place of the recursion.
- The resulting method is referred to as the *iterative version*.

Recursion vs. Iteration, cont.



- A recursive version of a method typically executes less efficiently than the corresponding iterative version.
- This is because the computer must keep track of the recursive calls and the suspended computations.
- However, it can be much easier to write a recursive method than it is to write a corresponding iterative method.

Recursion



- ▣ Recursion can describe everyday examples
 - Show everything in a folder and all its subfolders
 - show everything in top folder
 - show everything in each subfolder in the same manner

Recursive Methods That Return a Value

- A recursive method can be a `void` method or it can return a value.
- At least one branch inside the recursive method can compute and return a value by making a chain of recursive calls.

Method factorial()

```
public static int factorial(n) {  
    if (n == 0) {  
        return 1;  
    }  
    else {  
        return n * factorial(n-1);  
    }  
}
```

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main()



```
int nfactorial = factorial(n);
```

Recursive invocation

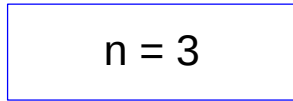
- A new activation record is created for every method invocation
- ▣ Including recursive invocations

main()



factorial()

n = 3



```
int nfactorial = factorial(n);
```

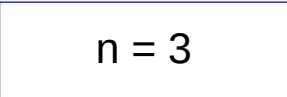
```
return n * factorial(n-1);
```

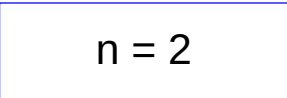


Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main() 

factorial() 

factorial() 

```
int nfactorial = factorial(n);
```

```
return n * factorial(n-1);
```

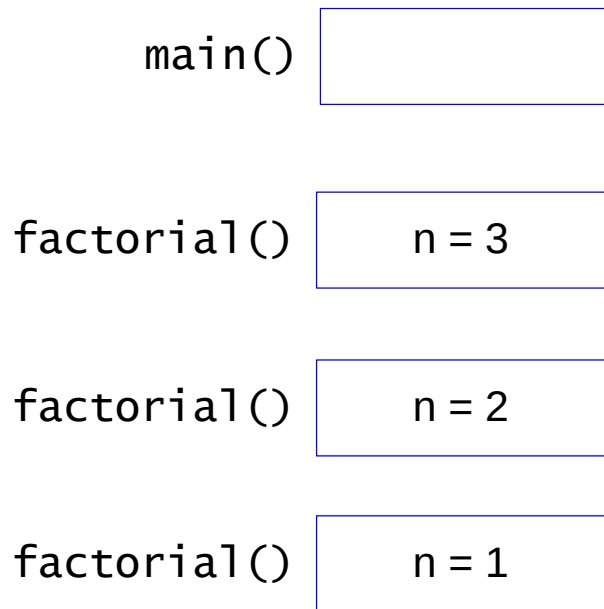


```
return n * factorial(n-1);
```



Recursive invocation

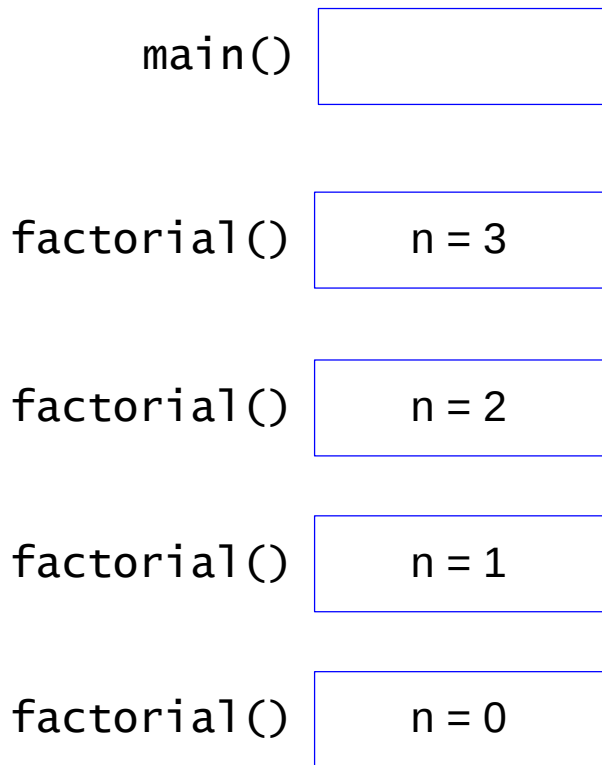
- A new activation record is created for every method invocation
 - ▣ Including recursive invocations



```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);
```

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations



```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return 1;
```

The diagram shows the sequence of recursive calls and return values for the factorial function. The code is as follows:

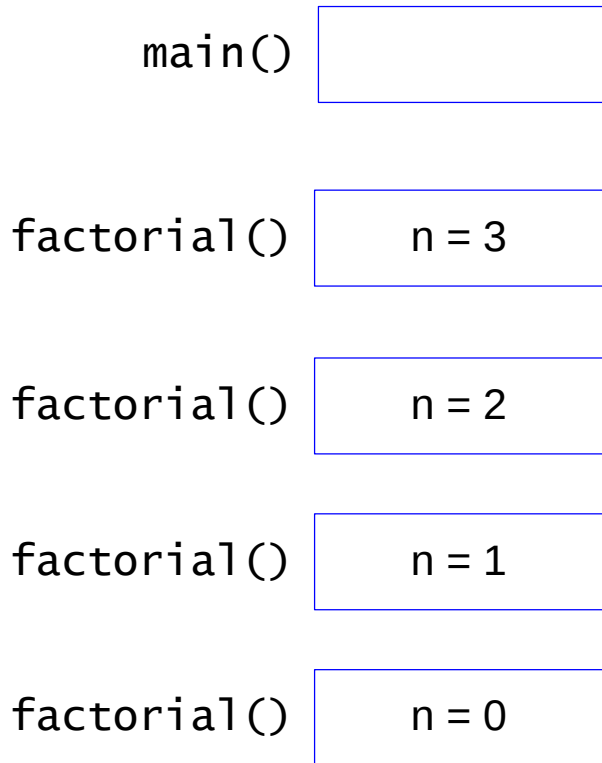
```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return 1;
```

Arrows indicate the flow of execution and return values:

- A pink arrow points from the `return 1;` line to the `return n * factorial(n-1);` line above it.
- A pink arrow points from the `return n * factorial(n-1);` line to the `return n * factorial(n-1);` line above it.
- A pink arrow points from the `return n * factorial(n-1);` line to the `return n * factorial(n-1);` line above it.
- A pink arrow points from the `return n * factorial(n-1);` line to the `int nfactorial = factorial(n);` line.

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

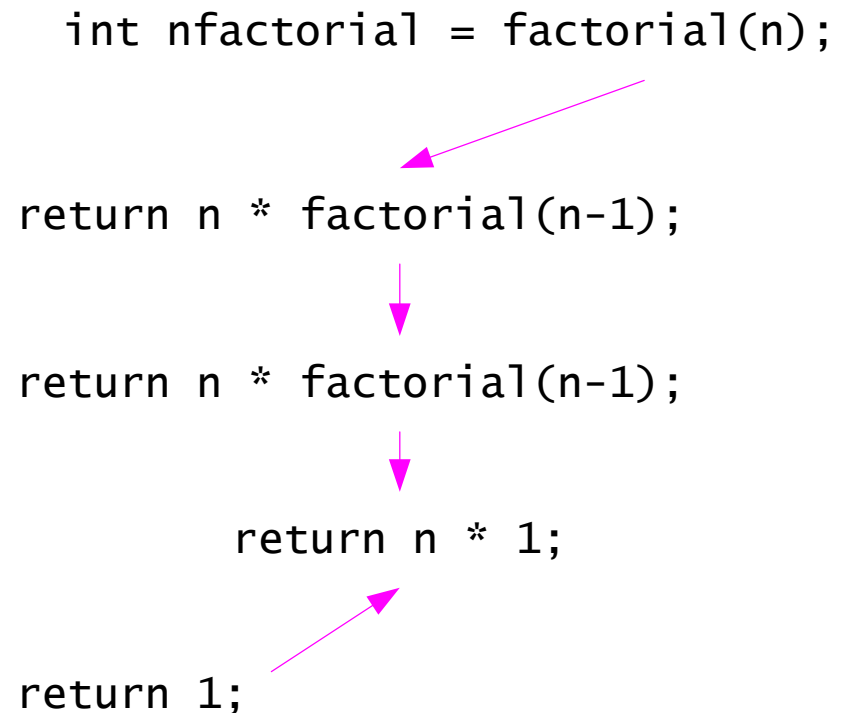
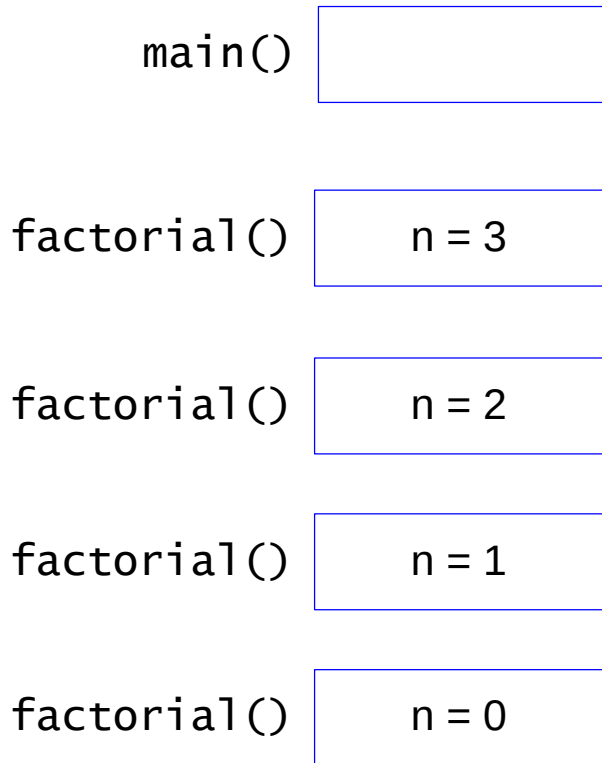


The diagram illustrates the recursive calls and return values for a factorial function. It shows a sequence of four `return n * factorial(n-1);` statements, with a final `return 1;` statement. Magenta arrows indicate the flow of control and return values: an arrow points from the `factorial(n-1)` in the first line to the `factorial(n-1)` in the second line; an arrow points from the `factorial(n-1)` in the second line to the `factorial(n-1)` in the third line; an arrow points from the `factorial(n-1)` in the third line to the `factorial(n-1)` in the fourth line; and an arrow points from the `return 1;` statement to the `factorial(n-1)` in the fourth line.

```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
return n * factorial(n-1);  
return n * factorial(n-1);  
return 1;
```

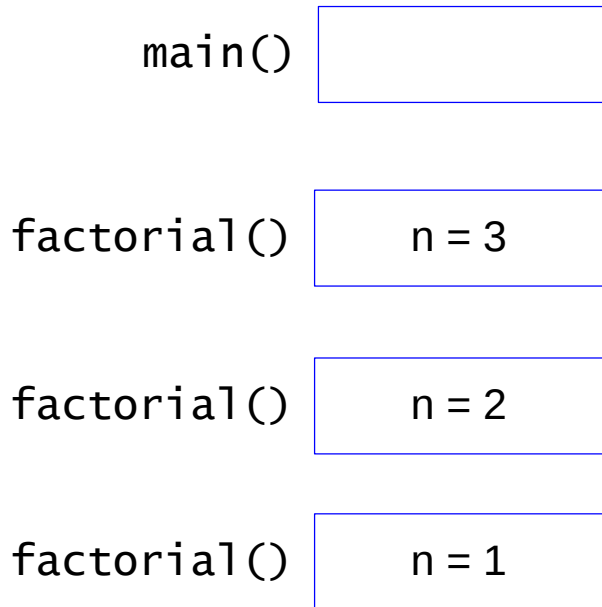
Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations



Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

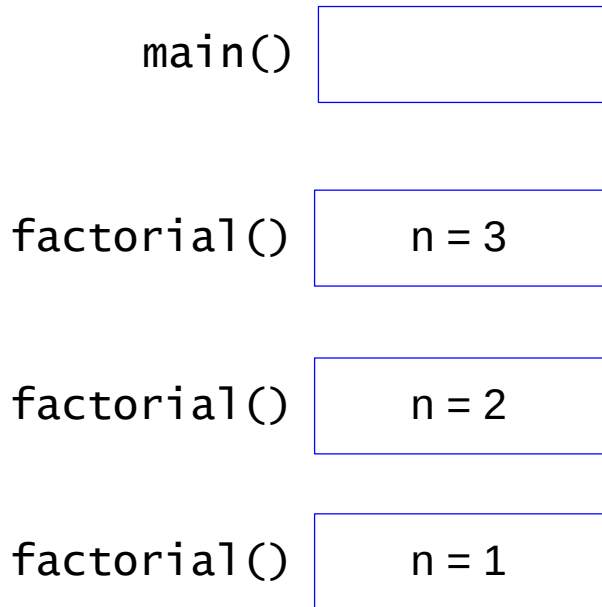


```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
  
return n * factorial(n-1);  
  
return 1 * 1;
```

Diagram illustrating the return flow of the recursive factorial function. Arrows indicate the return path from the base case up through the recursive calls.

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations



The diagram illustrates the return flow of a recursive factorial function. It shows the following code snippets with arrows indicating the flow of return values:

```
int nfactorial = factorial(n);
```

return n * factorial(n-1);

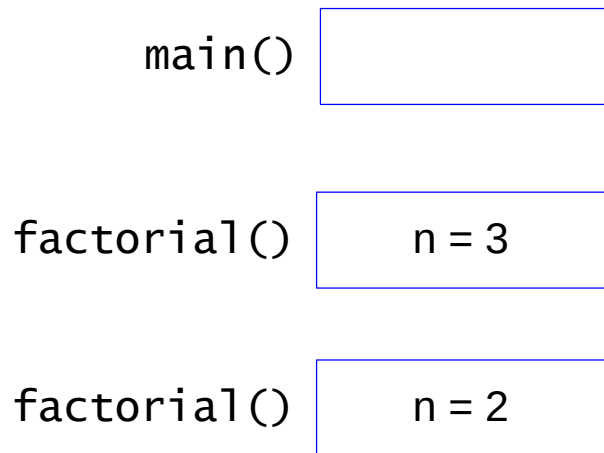
return n * 1

return 1 * 1;

The arrows show the return values being passed back up the call stack: from the base case 'return 1 * 1;' to 'return n * 1', then to 'return n * factorial(n-1);', and finally to the initial call 'int nfactorial = factorial(n);'.

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

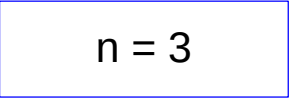


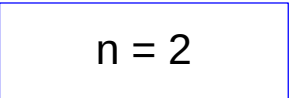
```
int nfactorial = factorial(n);  
  
return n * factorial(n-1);  
  
return 2 * 1
```

Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main() 

factorial() 

factorial() 

```
int nfactorial = factorial(n);
```

return n * 2;

return 2 * 1;

Recursive invocation

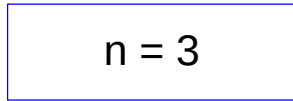
- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main()



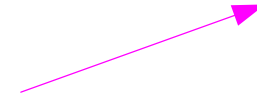
factorial()

n = 3



```
int nfactorial = factorial(n);
```

```
return 3 * 2;
```



Recursive invocation

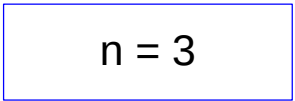
- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main()



factorial()

n = 3



```
int nfactorial = 6;
```

```
return 3 * 2;
```



Recursive invocation

- A new activation record is created for every method invocation
 - ▣ Including recursive invocations

main()

int nfactorial = 6;

Infinite recursion

- A common programming error when using recursion is to not stop making recursive calls.
 - ▣ The program will continue to recurse until it runs out of memory.
 - ▣ Be sure that your recursive calls are made with simpler or smaller subproblems, and that your algorithm has a base case that terminates the recursion.

Fibonacci Series Code

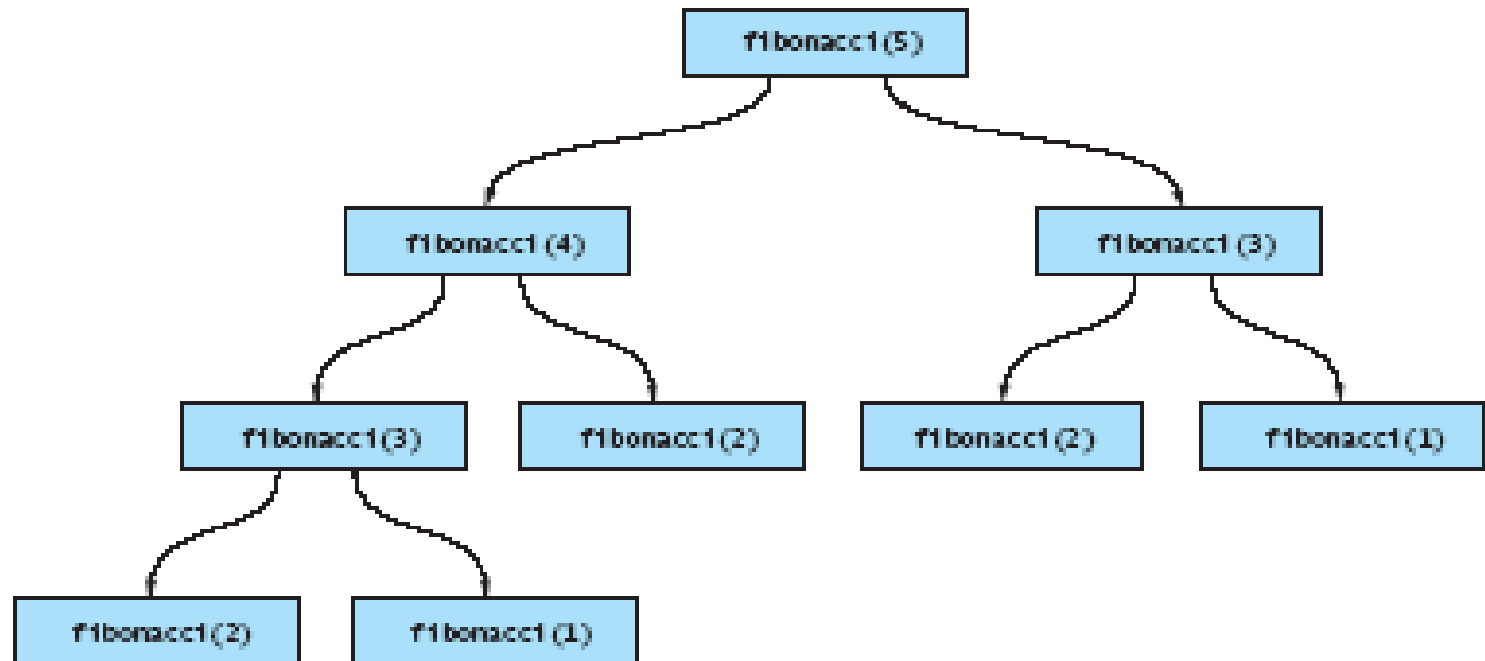
```
public static int fib (int n) {  
    if (n <= 2)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

This is straightforward, but an inefficient recursion ...

1, 1, 2, 3, 5, 8, 13,.....

Efficiency of Recursion: Inefficient Fibonacci

FIGURE 7.6
Method Calls Resulting
from fibonacci(5)



Efficient Fibonacci

- **Strategy:** keep track of:
 - Current Fibonacci number
 - Previous Fibonacci number

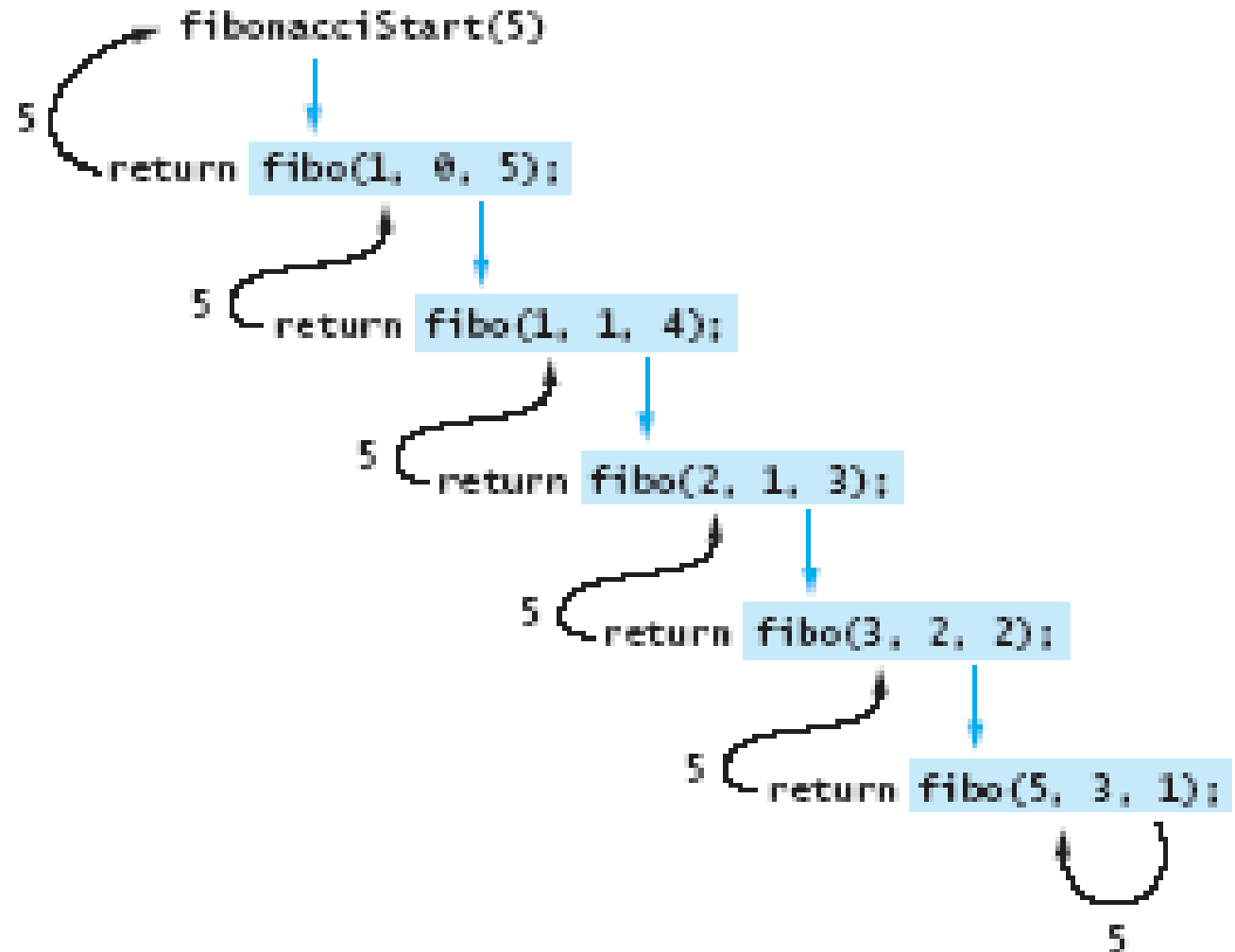
Efficient Fibonacci: Code

```
public static int fibStart (int n) {  
    return fibo(1, 1, n);  
}  
  
private static int fibo (  
    int curr, int prev, int n) {  
    if (n <= 1)  
        return curr;  
    else  
        return fibo(curr+prev, curr, n-1);  
}
```

Efficient Fibonacci: A Trace

FIGURE 7.7

Trace of
fibonacciStart(5)



Recursion Advantages

- Expressive Power
 - ▣ Recursive code is typically much shorter than iterative.
- More appropriate for certain problems.
- Intuitive programming from mathematic definitions.

Recursion Disadvantages

- Usually slower due to call stack overhead.
- Faulty Programs are very difficult to debug.
- Difficult to prove a recursive algorithm is correct



End